

INTRODUCTION TO KUBERNETES AND ORCHESTRATION

❖ Introduction to Kubernetes

- **Kubernetes** (also known as **K8s**) is an **open-source container orchestration platform** designed to automate the **deployment, scaling, and management** of **containerized applications**.

➤ Key Functions of Kubernetes:

1. Container Orchestration

Manages containers (like those created with Docker) across multiple hosts.

2. Automatic Scaling

Increases or decreases application instances based on demand.

3. Self-Healing

Automatically restarts failed containers, replaces them, and kills containers that don't respond to health checks.

4. Load Balancing & Service Discovery

Distributes network traffic and exposes containers using services.

5. Automated Rollouts and Rollbacks

Smooth deployment of updates while allowing rollback if something breaks.

6. Storage Management

Mounts and manages storage systems (local, cloud, or network).

▪ Basic Components of Kubernetes:

Component	Description
Pod	The smallest unit, usually holding one or more containers.
Node	A worker machine (VM or physical) where Pods run.
Cluster	A set of nodes managed by Kubernetes.
Master (Control Plane)	Coordinates the cluster (includes scheduler, API server, controller manager).
Deployment	Manages a set of identical Pods with auto-replacement and scaling.
Service	Defines a network policy to access a set of Pods (load balancing).

❖ Why Use Kubernetes?

Kubernetes solves many real-world problems that arise when deploying and managing modern applications, especially at scale.

❖ Architecture of Kubernetes

Kubernetes follows a Master-Worker architecture.

1. Control Plane (Master Components):

- **kube-apiserver** – Central management and communication hub
- **etcd** – Key-value store for configuration data
- **kube-scheduler** – Assigns pods to nodes
- **kube-controller-manager** – Handles controllers (replica, node, etc.)
- **cloud-controller-manager** – Integrates with cloud providers

2. Node Components (Worker Nodes):

- **kubelet** – Agent on each node to manage pod lifecycle
- **kube-proxy** – Manages networking rules and traffic routing
- **Container Runtime** – Runs containers (e.g., Docker, containerd)

❖ Lifecycle of a Pod

A **Pod** is the smallest deployable unit in Kubernetes and typically contains one or more containers.

➤ Pod Lifecycle Phases:

- **Pending** → Pod is accepted but not yet scheduled (*Pod accepted, waiting to be scheduled*)
- **Scheduled** → Pod is assigned to a node. (*Assigned to a node*)
- **Running** → Pod is initialized and containers are running (*Containers started*)
- **Succeeded** → Containers completed successfully (*Completed or terminated*)
- **Failed** → Containers terminated with failure
- **Unknown** → Status cannot be retrieved due to communication issues (*Status cannot be determined*)

❖ Cluster Creation Methods

Different tools and services allow us to create and manage Kubernetes clusters

Method	Description
Minikube	Local single-node Kubernetes cluster for testing
Kind (Kubernetes IN Docker)	Runs Kubernetes clusters using Docker containers, used for CI
kubeadm	Tool for bootstrapping production-grade Kubernetes clusters
EKS (Elastic Kubernetes Service)	Managed Kubernetes service by AWS
GKE (Google Kubernetes Engine)	Managed Kubernetes service by Google Cloud
AKS (Azure Kubernetes Service)	Managed Kubernetes service by Microsoft Azure

[GENERAL INTERVIEW QUESTIONS]

Q1. What is Kubernetes?

Q2. Why do we need orchestration tools like Kubernetes?

Q3. What is the function of kube-proxy?

Q3. What are the main components of the Kubernetes architecture?

- **Control Plane:** kube-apiserver, etcd, controller manager, scheduler
- **Node Components:** kubelet, kube-proxy, container runtime

Q4. What is a Pod in Kubernetes?

- A pod is the smallest deployable unit that contains one or more containers sharing storage and network resources.

Q5. What is the role of etcd?

- etcd is a key-value store used by Kubernetes to store cluster state and configuration data.

Q6. What is the kubelet?

- kubelet runs on each node and ensures containers are running according to the pod specifications.

Q7. What is a Deployment in Kubernetes?

- A Deployment manages ReplicaSets and enables declarative updates for pods and containerized applications.

Q8. What is the difference between Minikube and kubeadm?

- **Minikube:**→ For local single-node clusters (for learning/dev)
- **kubeadm:**→For bootstrapping production-ready multi-node clusters

Q9. What is a ReplicaSet?

- A **ReplicaSet** ensures a specified **number of pod replicas** are running at all times.

Q10. What are Services in Kubernetes?

- Services **expose pods for internal/external** communication using stable virtual IPs and DNS names